

Gnani Audio Notes — Concept & Build Reference

Full-Stack / AI Engineer internship take-home · Gnani.ai · Prepared 24 Aug 2026 · Submission deadline Fri 28 Aug 2026, 23:59 IST

How to use this document. Part 3 is the spine — one realistic story that touches every technical concept in the project, in the exact order they occur. If you can retell that story out loud without notes, you can defend this project in an interview. Part 8 is the flashcard glossary for quick revision.

Contents

- 1 — What the assignment actually is

2 — Why Gnani set *this* assignment

3 — The real-life scenario, end to end

4 — ASR concepts specific to this company

5 — The 15 locked design decisions
- 6 — Assumptions baked into the build

7 — Timeline, blockers and cut list

8 — Glossary for quick revision

9 — Interview questions to rehearse

1 — What the assignment actually is

Strip away the jargon and it is this:

Someone gives your website a recording of a person talking. Your website gives back the words that were said, plus a short summary of them. And it remembers everything they have uploaded before.

Three deliverables to the user: **a transcript, a summary, a history**. Everything else in the plan exists to make those three things work reliably.

The brief also names four hard requirements:

- It must be **deployed** — a URL that works with no setup on the reviewer's machine.
- It must have an **/architecture page** explaining the system, linking the GitHub repo.
- It must **handle failures visibly** — upload errors, API timeouts, corrupt files.
- It must **show upload and processing progress**.

2 — Why Gnani set *this* assignment

ASR = Automatic Speech Recognition. Software that listens to audio and writes down the words. Also called **STT (Speech-To-Text)** — same thing, two names. Gnani's product is ASR, specifically for Indian languages. That is their entire company.

Now look at these two facts side by side.

The brief says	Gnani's API docs say
"User uploads an audio file (2+ min)"	Maximum duration: 60 seconds (ideal ≤ 30s)

This is not an accident. The assignment is impossible if you use their API the obvious way. They deliberately asked for something their own synchronous endpoint cannot do in a single call. The real question being asked is: *when you hit a wall, do you notice it, and what do you build?*

Everything else on their list — progress bars, error handling, the architecture page — checks whether you can build a **system** rather than a **script**. A script calls an API. A system handles the file being 12 minutes long, the API being down, the user closing the tab, and the file being a corrupted MP3 that is actually a renamed PDF.

The documented API contract

Field	Value
Endpoint	POST <code>https://api.vachana.ai/stt/v3</code>
Auth header	X-API-Key-ID: <code><your key></code>
Body	<code>multipart/form-data</code> — <code>audio_file</code> , <code>language_code</code>
Optional	<code>format</code> = <code>verbatim</code> (default) or <code>transcribe</code> (enables ITN)
Formats	WAV, MP3, OGG, FLAC, AAC, M4A — auto-converted to 16 kHz mono
Max duration	60 seconds
Model	Synchronous — one request, one response, no job polling
Response	<code>{ success, request_id, timestamp, transcript }</code>
Errors	<code>400</code> invalid, <code>429</code> rate limit, <code>500</code> internal, <code>503</code> unavailable
Languages	bn-IN, en-IN, gu-IN, hi-IN, kn-IN, ml-IN, mr-IN, pa-IN, ta-IN, te-IN

3 — The real-life scenario, end to end

Meet **Priya**. She is a sales rep. She has just finished a 7-minute call with a client and recorded it on her phone. She wants notes without listening again. Here is every single thing that happens from the moment she opens your site.

0 The two computers

There are always two computers in a web app.

- **The client** — Priya's phone, running your React code inside her browser. She can see it. She can open DevTools and read every line. **Never put a secret here.**
- **The server** — a computer in a data centre running your backend code. She cannot see inside it. Your Gnani API key lives here and only here.

Why this matters practically: if you called Gnani's API from the browser, your API key would be visible to anyone who opens DevTools, and they would spend your Rs. 1,000 of credits. So every call to Gnani goes from *your server*, never from the browser.

1 She drops the file in

She picks **English (India)** from the dropdown and drags in `call_recording.m4a` — 7 minutes, 6 MB.

Why a language dropdown?

Gnani's API has a **required** parameter called `language_code`. It does not guess. You must tell it "this audio is Hindi" or "this audio is Tamil". The codes look like `hi-IN`, `ta-IN`, `en-IN` — that format is a standard called **BCP-47**. `hi` is the language, `IN` is the region. So `en-IN` means "English as spoken in India", which is a genuinely different acoustic model from American English — trained on Indian accents.

2 Storage, and why there are two kinds

Your app stores two completely different kinds of thing.

	Vercel Blob (object storage)	Neon Postgres (database)
Stores	Big files — the actual audio bytes	Small structured facts — "note 47 is 7 min long, status = processing"
Good at	Holding megabytes cheaply	Answering questions like "list Priya's notes, newest first"
Bad at	Being queried	Holding megabytes

Blob literally means Binary Large OBject. Object storage is a hard drive on the internet: you hand it a file, it hands you back a URL. It cannot answer "which files are longer than 5 minutes" — that is what the database is for.

So: the audio goes in Blob, the facts about the audio go in Postgres. The database stores the Blob's URL as a text column, linking the two.

The upload itself

Priya's browser sends the file **directly** to Blob. Your server only issues a short-lived permission slip (a "client token") saying "this browser may upload one file". The 6 MB never touches your server.

Two reasons for doing it that way:

1. Your server is not tied up for 40 seconds shovelling bytes.
2. The browser upload API gives you an `onUploadProgress` event that fires as bytes leave the phone. **That is how you draw a real progress bar.** The standard `fetch()` function gives you nothing while uploading — you would have no idea whether it was at 5% or 95%.

3 The job is created

Upload finishes. The browser tells your server: "done, here is the Blob URL, language is en-IN."

Your server writes a row to the database:

```
notes: id=47, status='pending', language='en-IN', blob_url='https://...'
```

...and **immediately replies** `202 Accepted` with `{ id: 47 }`.

HTTP status codes you should know cold

Code	Means	Where you will see it
200 OK	Worked	A successful transcript
202 Accepted	"I have taken your request, I will work on it"	Your job creation endpoint
400 Bad Request	You sent something invalid	Gnani rejects a bad audio format
401 Unauthorized	Bad or missing API key	Wrong <code>X-API-Key-ID</code>
429 Too Many Requests	You are going too fast, slow down	Gnani rate-limiting your fan-out
500 Internal Server Error	Their code broke	A Gnani hiccup
503 Service Unavailable	They are down or overloaded	Gnani maintenance

Why 202 and not 200?

Because transcribing 7 minutes of audio takes roughly 60 seconds, and an HTTP request that hangs for 60 seconds will be killed by browsers, proxies and load balancers. So you do not make her wait on the request. You say "got it, job 47", close the connection, and do the work separately.

Term	Meaning	Analogy
Synchronous	Ask, and wait on the line for the answer	A phone call
Asynchronous	Ask, hang up, get told later	Sending a text

Gnani's `/stt/v3` endpoint is synchronous. **Your** job endpoint is asynchronous.

4 Validation, before you spend a rupee

The job starts. The first thing it does — before touching Gnani — is run **ffprobe**.

ffmpeg is the universal audio and video tool. It converts, cuts, merges and analyses almost any media format. It is a command-line program — you run it with arguments, not by importing a library. **ffprobe** is its inspector twin: it changes nothing, it only reports *what a file is*.

You run ffprobe and it reports: duration 7m 12s, codec AAC, 44.1 kHz, stereo. That gives you three things:

1. **Corrupt-file detection.** If someone renames `invoice.pdf` to `audio.mp3`, ffprobe fails immediately with "Invalid data found when processing input". You fail the job in 200 ms with a clear message, instead of sending garbage to Gnani and burning credits on 14 failed calls.
2. **True duration.** The browser lies about this or does not know. ffprobe is authoritative.
3. **Enforcing the 10-minute cap.** A 45-minute podcast gets rejected here, cheaply, before any API spend.

This is called a **pre-flight check** — validate before committing to expensive work. Same idea as checking you have your passport before driving to the airport.

5 Chunking — the heart of the whole thing

7m 12s = 432 seconds. Gnani accepts 60. You need pieces.

The naive approach, and why it is bad

Cut every 30 seconds: `ffmpeg -f segment -segment_time 30`. Fifteen pieces, one command. But at exactly 30.000 seconds Priya was mid-sentence:

```
"...so the revenue pro | jection for Q3 looks strong..."
```

The word `projection` is sliced in half. The ASR hears half a word in each chunk and produces garbage at every seam. In a 7-minute file that is **14 damaged spots**, and a reviewer running one demo will see all of them.

Silence-aware splitting

Run ffmpeg's `silencedetect` filter first. It scans the audio and reports every moment quieter than a threshold for longer than, say, 0.3 seconds — every natural pause between sentences:

```
silence_start: 24.8    silence_end: 25.4
silence_start: 51.2    silence_end: 51.9
...
```

Then instead of cutting blindly at 25 s, you cut at **the nearest silence to the 25 s mark** — 24.8 s here. Nobody was speaking. No word is damaged. If there is no silence within ± 5 s (someone talking non-stop), you cut at 30 s anyway, because 30 is the hard ceiling.

This costs one extra ffmpeg pass and some text parsing. It is the single most technically interesting thing in the entire project, and it is exactly what **"long-audio handling"** means on the architecture page.

Result: 15 chunks, roughly 25–29 s each, each cut at a pause. You write 15 rows to the `chunks` table:

```
chunks: note_id=47, idx=0, start_ms=0,      end_ms=24800, status='pending'
        note_id=47, idx=1, start_ms=24800, end_ms=51200, status='pending'
        ...
```

You keep `idx` because **the pieces must be reassembled in order** — chunk 7's text has to land between 6 and 8. You keep `start_ms` and `end_ms` because if chunk 7 fails, you can tell the user exactly which 26 seconds of their recording are missing.

6 Fan-out, concurrency and retries

Fan-out

Fan-out = taking one task and splitting it into many parallel sub-tasks. Picture one arrow coming in and fifteen arrows going out. The reverse — collecting the fifteen results back into one — is called **fan-in**.

The alternative is **sequential**: chunk 0, wait, chunk 1, wait, chunk 2... At roughly 3 seconds each that is 45 seconds. Fan-out does them at the same time.

Concurrency

Concurrency = how many things you allow to be in progress simultaneously.

"Concurrency 4" means at most 4 requests to Gnani are in flight at any moment. When one finishes, the next chunk starts. Like a shop with 4 checkout counters and 15 customers — 4 are being served, the rest wait in line, and it stays exactly 4 until the queue empties.

Question	Answer
Why not fire all 15 at once?	Gnani sends back <code>429 Too Many Requests</code> . Their docs list <code>429</code> as a real error but publish no actual rate limit, so you are flying blind — start conservative.
Why not 1 (sequential)?	45 seconds of staring at a screen versus ~12 seconds is a real product difference.

Net effect: 15 chunks, 4 at a time, roughly 12 seconds total.

Retries and exponential backoff

Networks fail. APIs hiccup. Chunk 7 comes back `500`.

Retry = try again. "3 retries" = try up to 3 more times before giving up.

But *when* do you retry? If you retry instantly and the reason for failure was that Gnani is overloaded, you have just added more load to an overloaded server. So you wait, and you wait longer each time:

Attempt	Wait before it
1st retry	1 second
2nd retry	2 seconds
3rd retry	4 seconds

Each wait is double the last. That doubling is why it is called **exponential backoff**. It gives a struggling service room to recover instead of stampeding it.

In production you would also add **jitter** — a small random offset — so that if 100 clients all fail at once they do not all retry at the same instant. Worth knowing the word; overkill for this project.

Which errors do you retry?

Error	Action	Why
<code>429 / 500 / 503</code>	Retry	Transient — it might work in a second
<code>400 Bad Request</code>	Do not retry	Permanent — the audio format is wrong and will be wrong forever. Retrying wastes 3 calls to learn nothing.

Knowing the difference between a **transient** failure and a **permanent** one is the whole skill here. Retry the first kind, surface the second immediately.

After the retries are exhausted

Chunk 7 has failed 4 times. Two philosophies:

- **All-or-nothing:** the note failed. Throw away the 14 successful transcriptions and the credits spent on them.
- **Graceful degradation:** keep the 14, insert a marker where 7 should have been, tell the user clearly.

We chose graceful degradation. Priya's transcript reads:

```
...discussed the migration timeline.  
[transcription unavailable: 3:04-3:31]  
...and she confirmed the budget was approved.
```

The note is marked `completed_with_errors`, a banner says "1 of 15 segments failed", and a **Retry failed segments** button re-runs only chunk 7 — which is cheap, because the `chunks` table knows exactly which one broke and where it starts.

She got 93% of her value instead of 0%. **That is what "handle failures visibly" means:** the failure is *shown*, not hidden, and not fatal.

7 Stitching

All chunks done. Fetch them ordered by `idx`, join the text, and you have the full transcript. This is the **fan-in**.

8 The summary

The transcript (~1,100 words for 7 minutes) goes to an **LLM (Large Language Model)** — ChatGPT-style AI. Gnanini does not sell one; they do speech, not text generation. So this comes from elsewhere.

You send a **prompt** — an instruction plus the transcript — and ask for **structured output**. That last part matters. If you just ask "summarise this", you get a paragraph of prose you would have to parse with string manipulation. Instead you define a **schema** (using Zod, a TypeScript library for describing data shapes):

```
{ title: string, tldr: string, keyPoints: string[], actionItems: string[] }
```

and the AI SDK's `generateObject()` guarantees you get back exactly that shape, validated, or it throws. No parsing, no "sometimes it wrapped the answer in markdown".

The title is the sneaky-valuable field. Without it, Priya's history page is a list of `WhatsApp Audio 2026-08-24 at 14.32.m4a`. With it, it is "Acme Corp — Q3 renewal discussion". That one field is the difference between a demo and a product.

Vercel AI Gateway is a middleman: instead of integrating with Anthropic and OpenAI and Google separately, you send everything to the Gateway with a model name like `"anthropic/claude-sonnet-4-5"` and it routes for you. One API key instead of three, plus **model fallback** — if the primary model is down it automatically tries a backup. That fallback is a direct, honest answer to the brief's "API timeouts" requirement.

9 Meanwhile — what has Priya been seeing?

This whole time, roughly 75 seconds, Priya watches a progress display. How does it know?

Polling. Her browser asks your server "what is the status of job 47?" every 2 seconds. Your server reads the database and answers. Ask, wait 2 s, ask again. Like refreshing a package tracking page, automatically.

```
GET /api/jobs/47 → { status: 'transcribing', done: 7, total: 15 }
```

Option	How it works	Why not here
Polling (chosen)	Client asks repeatedly	Simple, survives a page refresh, works everywhere
SSE (Server-Sent Events)	Server holds a connection open and pushes updates	The work runs in a <i>different</i> server invocation, so an SSE endpoint would have to poll the database internally anyway — same polling, more complexity
WebSockets	Two-way persistent connection	Massive overkill — you have no two-way traffic

Because progress lives in **Postgres** and not in browser memory, Priya can close her laptop, reopen it, and the page picks up at "Transcribing 11/15". That is a real feature you get free from putting state in the database.

What she actually sees — the state machine

```
Uploading 62% → Validating audio → Splitting into 15 segments
               → Transcribing 7/15 → Summarizing → Done
```

Every one of those numbers is **measured**, not animated. `7/15` is a literal `COUNT(*) WHERE status='done'` from your chunks table. You earned that number by chunking — a candidate who sent one big file to a batch API can only show a spinner.

10 What if the server dies mid-job?

Serverless functions can be killed. Deploys happen. Things crash. If your job dies at chunk 9, note 47 sits at `status='processing'` forever and Priya's page spins until she gives up.

So: a **stale-job reaper**. A **cron job** (a task that runs automatically on a schedule) fires every 5 minutes and runs one query — "any note still `processing` that started more than 15 minutes ago?" — and marks those `failed` with "processing timed out".

Small piece of code, big signal. It is the difference between "I built the happy path" and "I thought about what happens when it breaks."

11 The history

Priya returns tomorrow. How does your app know which notes are hers, with no login?

A **cookie** — a small piece of data the server asks the browser to store and send back on every future request. On her first visit you generate a random ID and set it as an **httpOnly cookie** (`httpOnly` means JavaScript on the page cannot read it, only the server can — this protects against certain attacks). Every note she creates is tagged with that ID. Her list is `SELECT * FROM notes WHERE session_id = <her cookie>`.

No signup, no password, and the reviewer can use the app in 3 seconds. The honest limitation — clear your cookies and your notes are orphaned — goes on the architecture page.

4 — ASR concepts specific to this company

Things that will come up if you work at an ASR company.

Concept	What it means
Sample rate	Audio is captured by measuring air pressure many times per second. 16,000 times/sec = 16 kHz. Phone calls are 8 kHz, music is 44.1 kHz. Gnani auto-converts everything to 16 kHz mono — the standard for speech models, because human speech lives below 8 kHz and higher rates add data without adding information.
Mono vs stereo	One channel vs two. Speech models want mono; stereo gets averaged down.
ITN Inverse Text Normalization	Converting spoken forms to written forms: "twenty five" → "25", "january third" → "Jan 3". Gnani's <code>format</code> parameter controls this — <code>verbatim</code> writes exactly what was said including "um"; <code>transcribe</code> enables ITN. For notes and summaries, ITN reads far better. Core ASR vocabulary.
Diarization	Figuring out <i>who</i> spoke <i>when</i> — "Speaker 1 said this, Speaker 2 said that". Gnani's <code>/stt/v3</code> response has no diarization; it returns a flat string. That is why speaker labels are impossible in this project and belong in "what I would improve", rather than having the LLM invent them.
WER Word Error Rate	The standard accuracy metric for ASR — what percentage of words were wrong. This is <i>the</i> number ASR companies compete on. Knowing this term signals you understand their industry.
Code-switching	Mixing languages mid-sentence — "मैंने usko call किया but he didn't pick up." Extremely common in India and genuinely hard for ASR. A big part of why Gnani exists as an Indian ASR company rather than everyone just using an off-the-shelf model.

5 — The 15 locked design decisions

#	Decision	Choice
1	Long audio	ffmpeg chunking + fan-out against the synchronous <code>/stt/v3</code> . No dependency on the undocumented batch API.
2	Hosting	Vercel · Next.js 16 App Router · Neon Postgres · Vercel Blob
3	Background work	<code>waitUntil</code> + status rows in Postgres + client polling, with a Vercel Cron reaper for stuck jobs
4	Upload path	Browser → Blob directly via client token; <code>onUploadProgress</code> for real byte-level percentage
5	Chunking	Silence-aware — <code>silencedetect</code> , cut at nearest silence to ~25 s, hard cap 30 s
6	Chunk failure	Partial transcript + timestamped gap marker + <code>completed_with_errors</code> + "retry failed segments"
7	LLM	AI SDK <code>generateObject</code> via Vercel AI Gateway, with model fallback
8	Identity	Anonymous httpOnly cookie session + 2–3 seeded public demo notes
9	Language	Dropdown, all 10 BCP-47 codes, default <code>en-IN</code>
10	Limits	10 minute / ~40 MB ceiling, enforced at ffprobe pre-flight, stated up-front in the UI
11	Progress	Full stage machine — upload % → validating → splitting into N → transcribing X/N → summarizing → done; refresh-resumable
12	Schema	<code>notes</code> + <code>chunks</code> tables
13	Summary	<code>{ title, tldr, keyPoints[], actionItems[] }</code> — title drives the notes list
14	<code>/architecture</code>	Constraint-driven narrative + sequence diagram + sync-vs-background table + honest limitations section + repo link
15	Sequencing	ffmpeg spike deployed Monday night → build locally → deploy Wednesday night → Thursday/Friday buffer

6 — Assumptions baked into the build

- ffprobe pre-flight is job step 0; nothing reaches ASR before the file is proven decodable.
- ASR fan-out concurrency **4**, 3 retries, exponential backoff on `429` / `500` / `503` .
- Client polls `/api/jobs/:id` every 2 s (not SSE — with `waitUntil` an SSE route would just poll Postgres for you).
- Summarization is **one call over the full stitched transcript**; map-reduce is documented as future work, not built.
- Note detail page shows an `<audio>` player against the Blob URL alongside transcript and summary.
- No test suite — four days, and nothing in the brief asks for one.
- Gap markers render inline as `[transcription unavailable: 3:04–3:31]` .

- No subagents. One coherent codebase, written and interrogated in one session, so the code is genuinely owned.

7 — Timeline, blockers and cut list

Blockers, in priority order

1. **Gnani API key — Monday night, before anything else.** Sign up at `gnani.ai/speech-to-text-api` , confirm the Rs. 1,000 credits are live, take the key from `app.gnani.ai/voice` . It gates all of Tuesday and the provisioning delay is outside your control. If it has not arrived by Tuesday morning, email Sabari.
2. **Record ~3 minutes of yourself talking.** You will know exactly what you said, which makes it the only test file where you can judge transcript accuracy and spot chunk-boundary damage. Record a second clip in an Indic language for the seeded demo note.
3. `npm i -g vercel` — the Vercel CLI is not installed; you will want `env pull` and `logs` .

Schedule

When	What
Mon night	Gnani signup (parallel) + throwaway Vercel route proving <code>ffmpeg-static</code> executes in production
Tuesday	Full pipeline: blob upload, schema, fprobe, silence split, ASR fan-out, stitching, summary
Wednesday	UI, progress state machine, failure states, seeded demos — deploy Wednesday night
Thursday	Architecture page, polish, mock-interview pass over the codebase
Friday	Buffer. Submit well before 23:59 IST.

If behind — cut in this order

1. Silence-aware split → degrade to fixed 30 s segments (one `ffmpeg` flag, ten minutes) and name it in "what I'd improve".
2. Retry-failed-segments → degrade to a full-note retry.
3. Seeded demo notes.

Never cut: the deployed URL, the `/architecture` page, visible failure handling, or the progress counter. Those four are named explicitly in the brief.

8 — Glossary for quick revision

Term	One-line definition
ASR / STT	Automatic Speech Recognition / Speech-To-Text — turning spoken audio into written words.
Client / server	The browser (visible to the user, no secrets) vs your backend (private, holds the API key).
API endpoint	A specific URL on a server that accepts requests and returns data.
multipart/form-data	The HTTP body format used to send files alongside text fields in one request.
Synchronous	Ask and wait on the connection for the answer. A phone call.
Asynchronous	Ask, disconnect, get told later. A text message.
202 Accepted	"I have your request and will work on it" — the correct reply when work outlives the request.
429	Too Many Requests — the server telling you to slow down.
Transient vs permanent	An error that might succeed on retry (429/500/503) vs one that never will (400).
Object storage / Blob	A hard drive on the internet. Give it a file, get back a URL. Cannot be queried.
Database	Structured, queryable facts. Holds the Blob URL, not the bytes.
ffmpeg	Command-line tool that converts, cuts and analyses audio and video.
ffprobe	ffmpeg's inspector — reports what a file is without changing it.
Pre-flight check	Validate cheaply before committing to expensive work.
silencedetect	An ffmpeg filter that reports every pause in the audio — the basis of clean chunk boundaries.
Chunking	Splitting one long input into many small ones that fit an API's limit.
Fan-out / fan-in	One task split into many parallel sub-tasks / collecting their results back into one.
Concurrency	How many operations you allow to be in flight at once. Four checkout counters, not fifteen.
Retry	Attempting a failed operation again.
Exponential backoff	Waiting twice as long before each successive retry — 1 s, 2 s, 4 s — so a struggling service can recover.
Jitter	A small random offset added to backoff so many clients do not retry in lockstep.
Idempotency	An operation safe to run twice with the same result — matters when retries might duplicate work.
Graceful degradation	Returning partial value on failure instead of nothing.
State machine	A fixed set of states and the allowed transitions between them — pending → processing → done / failed.
Polling	The client repeatedly asking "is it done yet?" on a timer.

SSE	Server-Sent Events — server holds a connection open and pushes updates one way.
WebSocket	A persistent two-way connection between browser and server.
Serverless function	Backend code that runs on demand per request, with no long-lived process you own.
waitUntil	Keeps a serverless invocation alive to finish background work after the response is sent.
Cron job	Code that runs automatically on a fixed schedule.
Stale-job reaper	A scheduled sweep that fails jobs stuck in "processing" past a deadline.
Cookie	Small data the server asks the browser to store and resend on every request.
httpOnly	A cookie flag meaning page JavaScript cannot read it — only the server can.
Environment variable	Configuration (like an API key) supplied to the server at runtime, never committed to git.
LLM	Large Language Model — the AI that writes the summary.
Structured output	Forcing the LLM to return a validated data shape instead of free prose.
Zod schema	A TypeScript way of describing and validating a data shape.
Model fallback	Automatically routing to a backup model when the primary is unavailable.
Map-reduce (summarising)	Summarising each chunk, then summarising the summaries — needed only for very long transcripts.
BCP-47	The language-code standard: <code>hi-IN</code> = Hindi, India.
Sample rate	Measurements of audio per second. 16 kHz mono is the speech-model standard.
ITN	Inverse Text Normalization — "twenty five" becomes "25".
Diarization	Identifying which speaker said what. Not available in Gnani's <code>/stt/v3</code> .
WER	Word Error Rate — the accuracy metric ASR companies compete on.
Code-switching	Mixing two languages within one sentence. Common in India, hard for ASR.

9 — Interview questions to rehearse

If you can answer these out loud without notes, you own this project.

1. Gnani's API takes 60 seconds. The brief asked for 2+ minutes. What did you do, and why that way?
2. Why cut on silence instead of every 30 seconds? What breaks if you do not?
3. Why concurrency 4? What happens at 15? What happens at 1?
4. Chunk 7 of 15 fails. Walk me through exactly what the user sees and what the database looks like.
5. Which HTTP errors do you retry and which do you not? Why?
6. Why exponential backoff rather than retrying immediately?
7. Why does your job endpoint return 202 instead of the transcript?
8. What runs synchronously and what runs in the background? Draw the boundary.
9. The server crashes at chunk 9. What happens to that note? Who notices, and how?

10. Why is the audio in Blob and not in Postgres?
 11. Why does the upload go browser → Blob instead of browser → your server → Blob?
 12. Why polling instead of WebSockets or SSE?
 13. Where does your Gnani API key live, and why can it never be in the client?
 14. A user uploads a renamed PDF. Trace what happens, step by step.
 15. How would you raise the 10-minute cap to 2 hours? What would have to change first?
 16. What are the three weakest parts of your system, and what would you build instead?
-